

AG35-CET&AG35-EUT

QuecOpen(SDK)设备管理指导

LTE Standard 模块系列

版本：1.0.1

日期：2024-03-13

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

文档历史

修订记录

版本	日期	作者	变更表述
-	2023-11-21	Jensen ZHANG	文档创建
1.0.0	2023-11-21	Jensen ZHANG	临时版本
1.0.1	2024-03-13	Jensen ZHANG	临时版本 1. 新增适用模块 AG35-EUT。 2. 新增如下函数（第 3.3 章）： ql_dm_set_radio_on() ql_dm_set_radio_off() ql_dm_set_service_error_cb() ql_dm_set_qoos_config() ql_dm_get_qoos_config() ql_dm_set_qoos_enable() ql_dm_get_qoos_enable() 3. 新增部分示例（第 4.12 章~第 4.18 章）。

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
2 设备管理飞行模式功能介绍.....	8
3 设备管理 API.....	9
3.1. 头文件.....	9
3.2. 函数概览	9
3.3. 函数详解	10
3.3.1. ql_dm_init.....	10
3.3.2. ql_dm_deinit.....	11
3.3.3. ql_dm_get_software_version	11
3.3.4. ql_dm_get_device_firmware_rev_id	12
3.3.5. ql_dm_get_modem_state	12
3.3.5.1. QL_DM_MODEM_STATE_TYPE_E.....	12
3.3.6. ql_dm_get_temperature	13
3.3.7. ql_dm_get_device_serial_numbers	13
3.3.7.1. ql_dm_device_serial_numbers_info_t	14
3.3.8. ql_dm_get_air_plane_mode	14
3.3.9. ql_dm_set_air_plane_mode.....	15
3.3.10. ql_dm_set_air_plane_mode_ind_cb.....	15
3.3.10.1. ql_dm_air_plane_mode_ind_cb.....	16
3.3.11. ql_dm_get_cpu_occupancy	16
3.3.12. ql_dm_get_mem_usage	17
3.3.13. ql_dm_set_modem_state_change_ind_cb.....	17
3.3.13.1. ql_dm_modem_state_ind_cb.....	17
3.3.14. ql_dm_set_radio_on.....	18
3.3.15. ql_dm_set_radio_off.....	18
3.3.16. ql_dm_set_service_error_cb.....	19
3.3.16.1. ql_dm_service_error_cb_f	19
3.3.17. ql_dm_get_qoos_enable.....	19
3.3.18. ql_dm_set_qoos_enable.....	20
3.3.19. ql_dm_set_qoos_config	20
3.3.20. ql_dm_get_qoos_config	21
4 示例	22
4.1. 获取 AP 侧软件版本号.....	22
4.2. 获取 modem 侧固件版本号	22
4.3. 获取 modem 状态	23
4.4. 获取模块晶振温度	23
4.5. 获取模块序列号	24

4.6.	获取模块飞行模式	24
4.7.	设置模块飞行模式	25
4.8.	设置飞行模式事件回调函数	25
4.9.	获取模块 AP 侧 CPU 占用率	26
4.10.	获取模块 AP 侧内存使用率	26
4.11.	设置 modem 状态改变回调函数	27
4.12.	开启射频	27
4.13.	关闭射频	28
4.14.	设置服务异常事件回调函数	28
4.15.	获取 QoS 使能状态	28
4.16.	设置 QoS 使能状态	29
4.17.	设置 QoS 配置参数	29
4.18.	获取 QoS 配置参数	30
5	附录 参考文档及术语缩写	31

Preliminary
Quectel Confidential

表格索引

表 1: 函数概览	9
表 2: 参考文档	31
表 3: 术语缩写	31

Preliminary
Quectel Confidential

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen®方案，主要介绍 AG35-CET 和 AG35-EUT 模块 SDK 中提供的设备管理 API 功能和使用方法。设备管理是终端管理业务，用于模块状态信息的查询及参数设定。

Preliminary
Quectel Confidential

2 飞行模式功能介绍

模块进入飞行模式时，会将射频电路断电，此时，模块不会进行任何无线信号的收发，甚至无法拨打紧急电话。模块退出飞行模式后，会恢复对射频电路的供电，模块可进行注网、接打电话、收发短信、拨号等业务。

无论飞行模式处于开启还是关闭状态，模块断电重启后都会退出飞行模式并恢复默认的飞行模式配置。

模块飞行模式，定义如下：

```
typedef enum QL_DM_AIR_PLANE_MODE_TYPE_ENUM
{
    QL_DM_AIR_PLANE_MODE_UNKNOWN    = 0,
    QL_DM_AIR_PLANE_MODE_ON         = 1,
    QL_DM_AIR_PLANE_MODE_OFF        = 2,
    QL_DM_AIR_PLANE_MODE_NA         = 3
}QL_DM_AIR_PLANE_MODE_TYPE_E;
```

● 成员

成员	描述
<code>QL_DM_AIR_PLANE_MODE_UNKNOWN</code>	飞行模式状态未知
<code>QL_DM_AIR_PLANE_MODE_ON</code>	飞行模式开启
<code>QL_DM_AIR_PLANE_MODE_OFF</code>	飞行模式关闭（默认状态）
<code>QL_DM_AIR_PLANE_MODE_NA</code>	飞行模式不可用

3 设备管理 API

3.1. 头文件

设备管理 API 头文件为 `ql_dm.h`，位于 SDK 的 `ql-sysroots/usr/include/ql-sdk/` 目录下。若无特殊说明，本文档所提及的头文件均在该目录下。

备注

设备管理 API 用于管理和控制模块设备，某些 API 会改变模块的行为，故用户需谨慎调用相关 API。

3.2. 函数概览

表 1：函数概览

函数	说明
<code>ql_dm_init()</code>	初始化设备管理服务
<code>ql_dm_deinit()</code>	去初始化设备管理服务
<code>ql_dm_get_software_version()</code>	获取 AP 侧软件版本号
<code>ql_dm_get_device_firmware_rev_id()</code>	获取 modem 侧固件版本号
<code>ql_dm_get_modem_state()</code>	获取 modem 状态
<code>ql_dm_get_temperature()</code>	获取模块晶振温度
<code>ql_dm_get_device_serial_numbers()</code>	获取模块序列号
<code>ql_dm_get_air_plane_mode()</code>	获取模块飞行模式
<code>ql_dm_set_air_plane_mode()</code>	设置模块飞行模式

<code>ql_dm_set_air_plane_mode_ind_cb()</code>	设置飞行模式事件回调函数
<code>ql_dm_get_cpu_occupancy()</code>	获取模块 AP 侧 CPU 占有率
<code>ql_dm_get_mem_usage()</code>	获取模块 AP 侧内存使用率
<code>ql_ms_dm_set_modem_state_change_ind_cb()</code>	设置 modem 状态改变回调函数
<code>ql_dm_set_radio_on()</code>	开启射频
<code>ql_dm_set_radio_off()</code>	关闭射频
<code>ql_dm_set_service_error_cb()</code>	设置服务异常事件回调函数
<code>ql_dm_get_qoos_enable()</code>	获取 QoS 的使能状态
<code>ql_dm_set_qoos_enable()</code>	设置 QoS 的使能状态
<code>ql_dm_get_qoos_config()</code>	获取 QoS 的配置参数
<code>ql_dm_set_qoos_config()</code>	设置 QoS 的配置参数

备注

若无特别说明，本文档所述函数均不支持并发调用，并且不能在相关回调函数中调用以上函数。

3.3. 函数详解

3.3.1. ql_dm_init

该函数用于初始化设备管理服务。

- 函数原型

```
int ql_dm_init(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败。错误码详见 `ql_type.h`。

备注

程序启动后，必须调用 `ql_dm_init()` 初始化设备管理服务

3.3.2. ql_dm_deinit

该函数用于去初始化设备管理服务。

- 函数原型

```
int ql_dm_deinit(void)
```

- 参数

无

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败。错误码详见 `ql_type.h`。

3.3.3. ql_dm_get_software_version

该函数用于获取 AP 侧软件版本号。该函数不要求设备管理服务必须初始化。

- 函数原型

```
int ql_dm_get_software_version(char *soft_ver, int soft_ver_len)
```

- 参数

`soft_ver`:
[Out] AP 侧软件版本号。

`soft_ver_len`:
[In] `soft_ver` 长度。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败。错误码详见 `ql_type.h`。

3.3.4. ql_dm_get_device_firmware_rev_id

该函数用于获取 modem 侧固件版本号。该函数在设备管理服务初始化成功后方可使用。

● 函数原型

```
int ql_dm_get_device_firmware_rev_id(char *firmware_rev_id, int firmware_rev_id_len)
```

● 参数

firmware_rev_id:

[Out] modem 侧固件版本号。

firmware_rev_id_len:

[In] *firmware_rev_id* 长度。单位：字节。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败。错误码详见 *ql_type.h*。

3.3.5. ql_dm_get_modem_state

该函数用于获取 modem 状态。该函数不要求设备管理服务必须初始化。

● 函数原型

```
int ql_dm_get_modem_state(QL_DM_MODEM_STATE_TYPE_E *modem_state)
```

● 参数

modem_state:

[Out] modem 状态；详见第 3.3.5.1 章。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败。错误码详见 *ql_type*。

3.3.5.1. QL_DM_MODEM_STATE_TYPE_E

modem 状态枚举定义如下：

```
typedef enum QL_DM_MODEM_STATE_TYPE_ENUM
```

```
{
    QL_DM_MODEM_STATE_UNKNOWN = 2,
    QL_DM_MODEM_STATE_ONLINE  = 1,
    QL_DM_MODEM_STATE_OFFLINE = 0
}QL_DM_MODEM_STATE_TYPE_E;
```

● 成员

成员	描述
<code>QL_DM_MODEM_STATE_UNKNOWN</code>	modem 状态未知
<code>QL_DM_MODEM_STATE_ONLINE</code>	modem 在线
<code>QL_DM_MODEM_STATE_OFFLINE</code>	modem 掉线

3.3.6. ql_dm_get_temperature

该函数用于获取模块晶振温度。模块中还有其他温度值，如芯片温度、功率放大器温度等。用户可直接访问 `/sys/class/thermal/thermal_zoneX/type` 文件查看温度类型。相同目录下的 `temp` 文件为各温度类型对应的温度值。该函数不要求设备管理服务必须初始化。

● 函数原型

```
int ql_dm_get_temperature(float *temperature)
```

● 参数

temperature:

[Out] 模块晶振温度。

● 返回值

`QL_ERR_OK` 函数执行成功

其他值 函数执行失败。错误码详见 `ql_type`。

3.3.7. ql_dm_get_device_serial_numbers

该函数用于获取模块的序列号。该函数在设备管理服务初始化成功后方可使用。

● 函数原型

```
int ql_dm_get_device_serial_numbers(ql_dm_device_serial_numbers_info_t *p_info)
```

● 参数

p_info:

[Out] 模块序列号信息；详见第3.3.7.1章。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败。错误码详见 *ql_type*。

3.3.7.1. ql_dm_device_serial_numbers_info_t

模块的序列号信息结构体定义如下：

```
typedef struct
{
    uint8_t imei_valid;
    char imei[QL_DM_IMEI_MAX_LEN + 1];
    uint8_t meid_valid;
    char meid[QL_DM_MEID_MAX_LEN + 1];
}ql_dm_device_serial_numbers_info_t;
```

● 参数

类型	参数	描述
uint8_t	<i>imei_valid</i>	<i>imei</i> 是否有效。 0 无效 非 0 有效
char	<i>imei</i>	获取的设备 IMEI 号；适用于 GSM/WCDMA 网络制式。
uint8_t	<i>meid_valid</i>	<i>meid</i> 是否有效。 0 无效 非 0 有效
char	<i>meid</i>	获取的设备 MEID 号；适用于 CDMA 网络制式。

3.3.8. ql_dm_get_air_plane_mode

该函数用于获取模块飞行模式。该函数在设备管理服务初始化成功后方可使用。

● 函数原型

```
int ql_dm_get_air_plane_mode(QL_DM_AIR_PLANE_MODE_TYPE_E *p_info)
```

- 参数

p_info:

[Out] 飞行模式信息；详见第2章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败。错误码详见 *ql_type*。

3.3.9. ql_dm_set_air_plane_mode

该函数用于设置模块飞行模式。飞行模式默认为关闭状态，配置断电后不保存。该函数在设备管理服务初始化成功后方可使用。

- 函数原型

```
int ql_dm_set_air_plane_mode(QL_DM_AIR_PLANE_MODE_TYPE_E air_plane_mode)
```

- 参数

air_plane_mode:

[In] 飞行模式；详见第2章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败。错误码详见 *ql_type*。

3.3.10. ql_dm_set_air_plane_mode_ind_cb

该函数用于设置飞行模式事件回调函数。该函数所做设置在程序退出后失效，如有需要，请重新调用该函数进行设置。设置成功后，若飞行模式状态发生变化，则会调用飞行模式事件回调函数。该函数在设备管理服务初始化成功后方可使用。

- 函数原型

```
int ql_dm_set_air_plane_mode_ind_cb(ql_dm_air_plane_mode_ind_cb cb_func)
```

- 参数

cb_func:

[Out] 飞行模式事件回调函数；详见第3.3.10.1章。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败。错误码详见 *ql_type*。

3.3.10.1. ql_dm_air_plane_mode_ind_cb

该回调函数处理飞行模式事件。

- 函数原型

```
typedef void (*ql_dm_air_plane_mode_ind_cb)(QL_DM_AIR_PLANE_MODE_TYPE_E
air_plane_mode)
```

- 参数

air_plane_mode:
[In] 模块飞行模式；详见第2章。

- 返回值

无

3.3.11. ql_dm_get_cpu_occupancy

该函数用于获取 AP 侧 CPU 占有率。该函数不要求设备管理服务必须初始化。

- 函数原型

```
int ql_dm_get_cpu_occupancy(float *cpu_occupancy)
```

- 参数

cpu_occupancy:
[Out] 当前 AP 侧 CPU 占有率。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败。错误码详见 *ql_type*。

3.3.12. ql_dm_get_mem_usage

该函数用于获取 AP 侧内存使用率。该函数不要求设备管理服务必须初始化。

- 函数原型

```
int ql_dm_get_mem_usage(float *mem_use)
```

- 参数

mem_use:

[Out] 当前 AP 侧内存使用率。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败。错误码详见 *ql_type*。

3.3.13. ql_dm_set_modem_state_change_ind_cb

该函数用于设置 modem 状态改变回调函数。设置断电不保存，设置成功后，若 modem 状态改变，则会调用该函数设置的回调函数。该函数在设备管理服务初始化成功后方可使用。

- 函数原型

```
int ql_ms_dm_set_modem_state_change_ind_cb(ql_dm_modem_state_ind_cb cb_func)
```

- 参数

cb_func:

[Out] modem 状态改变回调函数；详见第 3.3.13.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败。错误码详见 *ql_type*。

3.3.13.1. ql_dm_modem_state_ind_cb

该回调函数获取 modem 状态。

- 函数原型

```
typedef void (*ql_dm_modem_state_ind_cb)(int modem_state)
```

- 参数

modem_state:

[In] modem 状态；详见第3.3.5.1章。

- 返回值

无

3.3.14. ql_dm_set_radio_on

该函数用于开启射频。其作用与 **AT+CFUN=1** 相同。该函数在设备管理服务初始化成功后方可使用。

- 函数原型

```
int ql_dm_set_radio_on(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.15. ql_dm_set_radio_off

该函数用于关闭射频。其作用与 **AT+CFUN=0** 相同。该函数在设备管理服务初始化成功后方可使用。

- 函数原型

```
int ql_dm_set_radio_off(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.16. ql_dm_set_service_error_cb

该函数用于设置服务异常事件回调函数。该函数在设备管理服务初始化成功后方可使用。

- 函数原型

```
int ql_dm_set_service_error_cb(ql_dm_service_error_cb_f cb)
```

- 参数

cb:

[In] 服务异常事件回调函数。详见第3.3.16.1章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.16.1. ql_dm_service_error_cb_f

该回调函数处理服务异常事件。

- 函数原型

```
typedef void (*ql_dm_service_error_cb_f)(int error)
```

- 参数

error:

[In] 异常码。取值描述如下：

-1099 Service Abort

- 返回值

无

3.3.17. ql_dm_get_qoos_enable

该函数用于获取 QoS 使能状态。

- 函数原型

```
int ql_dm_get_qoos_enable(char *enable);
```

- 参数

enable:

[out] QoS 使能状态。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

备注

模块中 QoS 机制为开启状态，不可设置。该函数为兼容 QoS 机制而实现，无实际功能。

3.3.18. ql_dm_set_qos_enable

该函数用于设置 QoS 使能状态。

- 函数原型

```
int ql_dm_set_qos_enable(char enable);
```

- 参数

enable:

[in] QoS 使能状态。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

备注

模块中 QoS 机制为开启状态，不可设置。该函数为兼容 QoS 机制而实现，无实际功能。

3.3.19. ql_dm_set_qos_config

该函数用于设置 QoS 配置参数。该函数在设备管理服务初始化成功后方可使用。

- 函数原型

```
int ql_dm_set_qos_config(int p1, int p2, int p3)
```

● 参数

p1:

[In] 第一轮搜网间隔时间。范围：1~225；单位：秒。

p2:

[In] 第二轮搜网间隔时间。范围：1~225；单位：秒。

p3:

[In] 第三轮搜网间隔时间。范围：1~225；单位：秒。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.20. ql_dm_get_qos_config

该函数用于获取 QoS 配置参数。该函数在设备管理服务初始化成功后方可使用。

● 函数原型

```
int ql_dm_get_qos_config(int *p1, int *p2, int *p3)
```

● 参数

p1:

[Out] 第一轮搜网间隔时间。范围：1~225；单位：秒。

p2:

[Out] 第二轮搜网间隔时间。范围：1~225；单位：秒。

p3:

[Out] 第三轮搜网间隔时间。范围：1~225；单位：秒。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

4 示例

用户可自行通过`/ql-sdk/sample/test_sdk_api/m_dm.c`查看设备管理 API 的完整示例，本章所述代码示例为部分示例。程序启动后，必须调用 `ql_dm_init()`初始化设备管理服务。

4.1. 获取 AP 侧软件版本号

```
void item_ql_dm_get_software_version(void)
{
    int ret;
    char soft_ver[128] = {0};

    ret = ql_dm_get_software_version(soft_ver, sizeof(soft_ver));

    printf("ql_dm_get_software_version ret = %d, software version is %s\n", ret, soft_ver);
}
```

4.2. 获取 modem 侧固件版本号

```
void item_ql_dm_get_device_firmware_rev_id(void)
{
    int ret;
    char firmware_rev_id[QL_DM_FIRMWARE_REV_MAX_LEN + 1] = {0};

    ret = ql_dm_get_device_firmware_rev_id(firmware_rev_id, sizeof(firmware_rev_id));
    printf("ql_dm_get_device_firmware_rev_id ret = %d, device revision id is %s\n",
        ret, firmware_rev_id);
}
```

4.3. 获取 modem 状态

```
void item_ql_dm_get_modem_state(void)
{
    int ret;
    QL_DM_MODEM_STATE_TYPE_E modem_state = QL_DM_MODEM_STATE_UNKNOWN;

    ret = ql_dm_get_modem_state(&modem_state);
    if(QL_DM_MODEM_STATE_ONLINE == modem_state)
    {
        printf("ql_dm_get_modem_state ret = %d, modem state is ONLINE\n", ret);
    }
    else if(QL_DM_MODEM_STATE_OFFLINE == modem_state)
    {
        printf("ql_dm_get_modem_state ret = %d, modem state is OFFLINE\n", ret);
    }
    else
    {
        printf("ql_dm_get_modem_state ret = %d, modem state UNKNOWN\n", ret);
    }
}
```

4.4. 获取模块晶振温度

```
void item_ql_dm_get_temperature(void)
{
    int ret;
    float temperature = 0.0;

    ret = ql_dm_get_temperature(&temperature);

    printf("ql_dm_get_temperature ret = %d, temperature=%.2f\n", ret, temperature/1000.0);
}
```

4.5. 获取模块序列号

```
void item_ql_dm_get_device_serial_numbers(void)
{
    int ret;
    ql_dm_device_serial_numbers_info_t t_info;
    memset(&t_info, 0, sizeof(ql_dm_device_serial_numbers_info_t));

    ret = ql_dm_get_device_serial_numbers(&t_info);
    printf("ql_dm_get_device_serial_number ret = %d", ret);
    if(t_info.imei_valid)
    {
        printf(", imei is %s", t_info.imei);
    }
    if(t_info.imei2_valid)
    {
        printf(", imei2 is %s", t_info.imei2);
    }
    if(t_info.meid_valid)
    {
        printf(", meid is %s ", t_info.meid);
    }
    printf("\n");
}
```

4.6. 获取模块飞行模式

```
void item_ql_dm_get_air_plane_mode(void)
{
    int ret;
    char mode_info[16] = {0};
    QL_DM_AIR_PLANE_MODE_TYPE_E air_plane_mode;

    ret = ql_dm_get_air_plane_mode(&air_plane_mode);

    printf("ql_dm_get_air_plane_mode ret = %d, ", ret);
    if(internal_dm_get_air_plane_mode(air_plane_mode, mode_info, sizeof(mode_info)) == 0)
    {
        printf("unrecognized air plane mode:%d\n", air_plane_mode);
    }
}
```

```

    }
    else
    {
        printf("current air plane mode is %s\n", mode_info);
    }
}

```

4.7. 设置模块飞行模式

```

void item_ql_dm_set_air_plane_mode(void)
{
    int ret;
    int mode;
    QL_DM_AIR_PLANE_MODE_TYPE_E air_plane_mode;

    printf("please input air plane mode(1: ON, 2: OFF): ");
    ret = t_get_int(&mode);
    if(ret != 0)
    {
        printf("Invalid input\n");
        return;
    }
    air_plane_mode = mode;
    if(air_plane_mode != QL_DM_AIR_PLANE_MODE_ON && air_plane_mode !=
    QL_DM_AIR_PLANE_MODE_OFF)
    {
        printf("please input 1 or 2\n");
        return;
    }

    ret = ql_dm_set_air_plane_mode(air_plane_mode);
    printf("ql_dm_set_air_plane_mode ret = %d\n", ret);
}

```

4.8. 设置飞行模式事件回调函数

```

void item_ql_dm_set_air_plane_mode_ind_cb(void)
{
    int ret = 0;
}

```

```

int reg_flag = 0;

printf("please input air plane mode reg option: (0: unreg, other: reg): ");
ret = t_get_int(&reg_flag);
if(ret != 0)
{
    printf("Invalid input\n");
    return;
}

if(reg_flag)
{
    ret = ql_dm_set_air_plane_mode_ind_cb(dm_air_plane_mode_event_ind_cb);
}
else
{
    ret = ql_dm_set_air_plane_mode_ind_cb(NULL);
}
printf("ql_dm_set_air_plane_mode_ind_cb ret = %d\n", ret);
}

```

4.9. 获取模块 AP 侧 CPU 占用率

```

void item_ql_dm_get_cpu_occupancy(void)
{
    int ret;
    float cpu_use = 0.0;

    ret = ql_dm_get_cpu_occupancy(&cpu_use);

    printf("ql_dm_get_cpu_occupancy ret = %d, cpu_occupancy = %.2f%%\n", ret, cpu_use);
}

```

4.10. 获取模块 AP 侧内存使用率

```

void item_ql_dm_get_mem_usage(void)
{
    int ret;
    float mem_use = 0.0;
}

```

```
ret = ql_dm_get_mem_usage(&mem_use);

printf("ql_dm_get_mem_usage ret = %d, mem_usage = %.2f%%\n", ret, mem_use);
}
```

4.11. 设置 modem 状态改变回调函数

```
void item_ql_dm_set_modem_state_change_ind_cb(void)
{
    int ret = 0;
    int reg_flag = 0;

    printf("please input modem state reg option: (0: unreg, other: reg): ");
    ret = t_get_int(&reg_flag);
    if(ret != 0)
    {
        printf("Invalid input\n");
        return;
    }

    if(reg_flag)
    {
        ret = ql_dm_set_modem_state_change_ind_cb(dm_modem_state_change_ind_cb);
    }
    else
    {
        ret = ql_dm_set_modem_state_change_ind_cb(NULL);
    }
    printf("ql_dm_set_modem_state_change_ind_cb ret = %d\n", ret);
}
```

4.12. 开启射频

```
void item_ql_dm_set_radio_on(void)
{
    int ret;

    ret = ql_dm_set_radio_on();
}
```

```
printf("ql_dm_set_radio_on ret = %d\n", ret);

}
```

4.13. 关闭射频

```
void item_ql_dm_set_radio_off(void)
{
    int ret;

    ret = ql_dm_set_radio_off();
    printf("ql_dm_set_radio_off ret = %d\n", ret);
}
```

4.14. 设置服务异常事件回调函数

```
void item_ql_dm_set_service_error_cb(void)
{
    int ret = 0;

    printf("Start to item_ql_dm_set_service_error_cb : ");
    ret = ql_dm_set_service_error_cb(dm_service_error_cb);
    if(ret != QL_ERR_OK)
    {
        printf("failed, ret=%d\n", ret);
    }
    else
    {
        printf("successful\n");
    }
}
```

4.15. 获取 QooS 使能状态

```
void item_ql_dm_get_qoos_enable(void)
{
    int ret = 0;
```

```

int enable = 0;
ret = ql_dm_get_qoos_enable(&enable);
if(ret != 0)
{
    printf("error occur\n");
    return;
}
printf("enable is %d\n", enable);
if(enable == 1)
{
    printf("qoos customized is enable\n");
}

printf("ql_dm_set_qoos_config ret = %d\n", ret);
}

```

4.16. 设置 QooS 使能状态

```

void item_ql_dm_set_qoos_enable(void)
{
    int ret;
    int enable;
    printf("input 0/1\n");
    t_get_int(&enable);
    ret = ql_dm_set_qoos_enable(enable);
    if(ret != 0)
    {
        printf("error occur\n");
        return;
    }
    printf("ql_dm_set_qoos_config ret = %d\n", ret);
}

```

4.17. 设置 QooS 配置参数

```

void item_ql_dm_set_qoos_config(void)
{
    int ret;

```

```

int phase1,phase2,phase3;
printf("please input qoos customized param(3) three time");
t_get_int(&phase1);
t_get_int(&phase2);
t_get_int(&phase3);
ret = ql_dm_set_qoos_config(phase1, phase2, phase3);
printf("ql_dm_set_qoos_config ret = %d\n", ret);
}

```

4.18. 获取 QooS 配置参数

```

void item_ql_dm_get_qoos_config(void)
{
    int ret;
    int phase1,phase2,phase3;
    ret = ql_dm_get_qoos_config(&phase1, &phase2, &phase3);
    printf("ql_dm_get_qoos_config ret = %d\n",ret);
    printf("qoos param: %d, %d, %d\n",phase1, phase2, phase3);
}

```

5 附录 参考文档及术语缩写

表 2：参考文档

文档名称
[1] Quectel_AG35-CER_QuecOpen_快速开发指导

表 3：术语缩写

缩写	英文全称	中文全称
AP	Access Point/Application Processor	接入点/应用处理器
API	Application Programming Interface	应用程序接口
CPU	Central Processing Unit	中央处理器
CMDA	Code-Division Multiple Access	码分多址(利用码序列相关性实现的多址通信)
DM	Device Management	设备管理
GSM	Global System for Mobile Communications	全球移动通信系统
IMEI	International Mobile Equipment Identifier	国际移动设备识别码
IoV	Internet of Vehicles	车联网
MEID	Mobile Equipment Identifier	移动设备识别码
NVRAM	Non-Volatile Random Access Memory	非易失性随机访问存储器
SDK	Software Development Kit	软件开发包
WCDMA	Wideband Code Division Multiple Access	宽带码分多址